



**Shri Dharmasthala Manjunatheshwara College of Engineering and Technology,
Dharwad-580002**

Department of Information Science and Engineering



Big Data Analytics

(22UISC602)

Source: Arshdeep Bahga and Vijay Madisetti

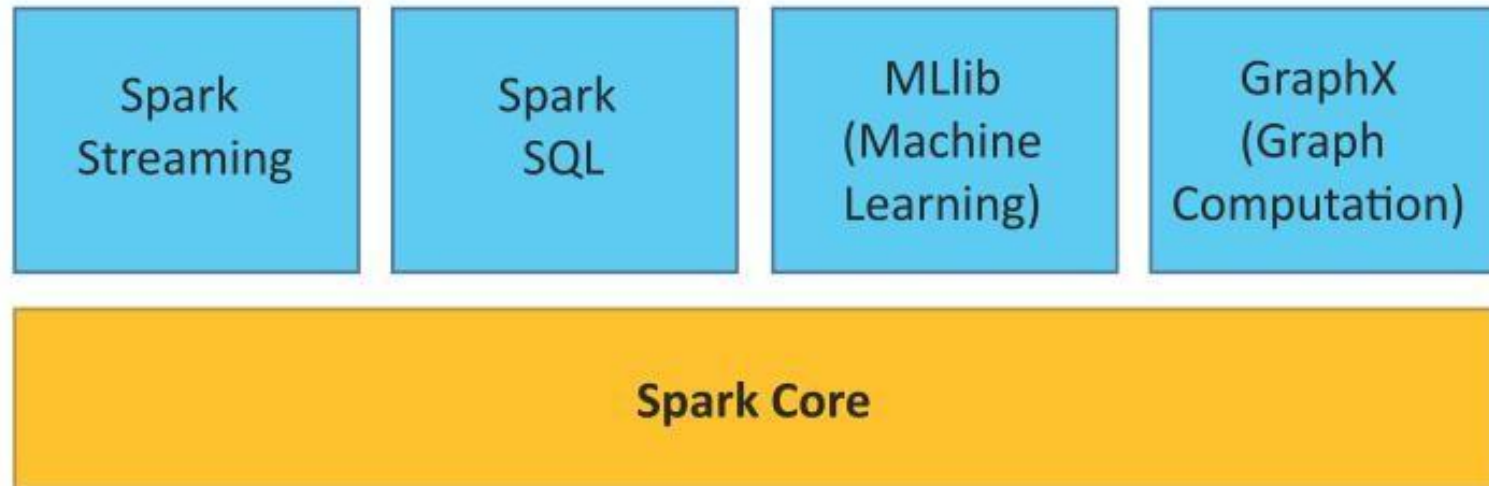
Course Instructor:
Dr. Rajashekarappa

Apache Spark

Source: Arshdeep Bahga and Vijay Madisetti

Spark

- Apache Spark is an open source cluster computing framework for data analytics
- Spark supports in-memory cluster computing and promises to be faster than Hadoop.
- Spark allows real-time, batch and interactive queries and provides APIs for Scala, Java and Python.



Spark Core

- Spark Core provides common functionality (such as task scheduling and input/output), which is used by other Spark components.
- Spark provides data abstraction called resilient distributed dataset (RDD) which is a collection of elements partitioned across the nodes of the cluster.
- The RDD elements can be operated on in parallel in the cluster.
- RDDs are immutable and distributed collection of objects.

Spark Streaming

- Spark Streaming is a Spark component for analysis of streaming data such as sensor data, click stream data, web server logs, etc.

Spark SQL

- Spark SQL is a Spark component that enables interactive querying of data using SQL queries

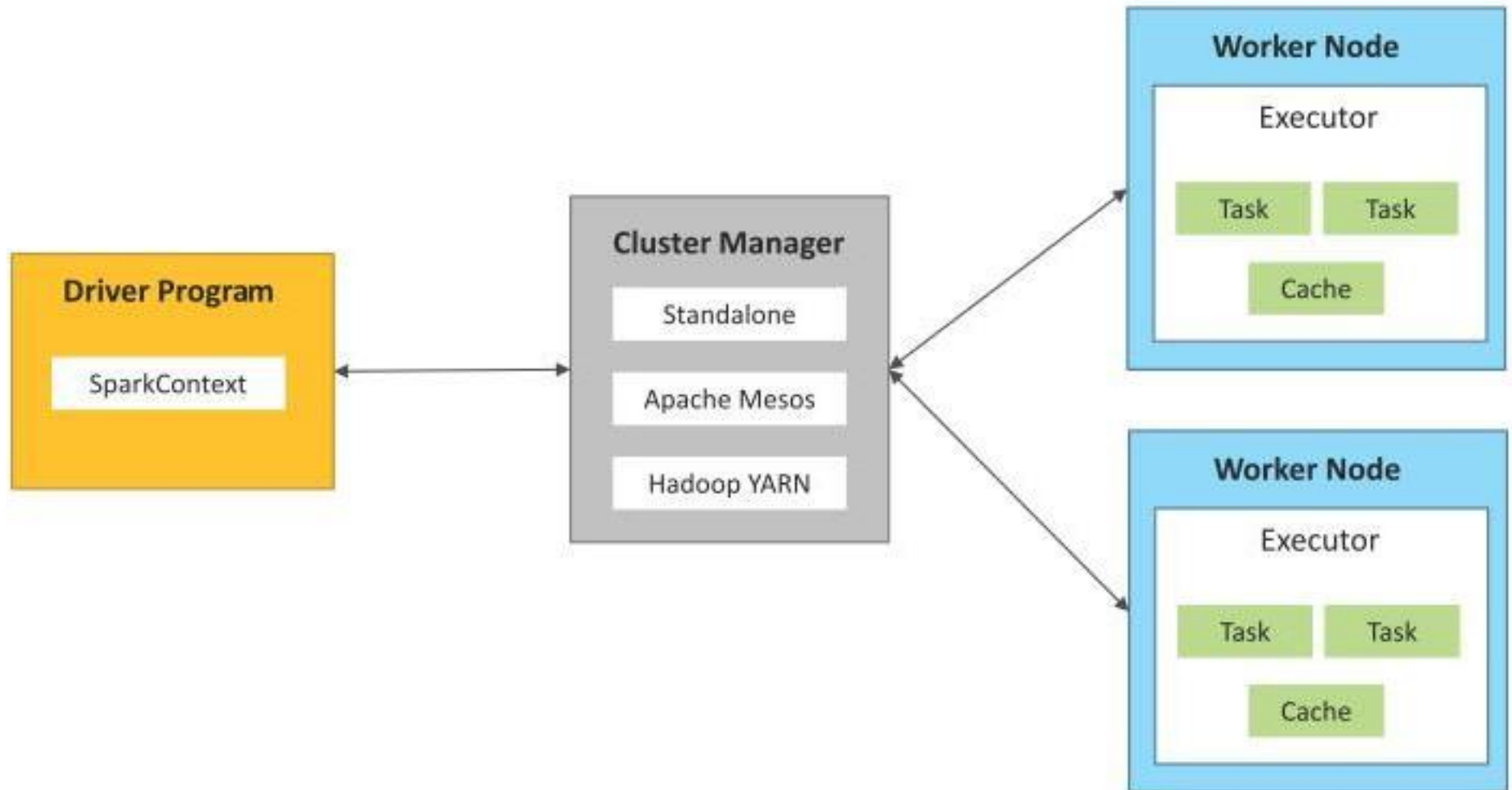
Spark MLlib

- Spark MLlib is Spark's machine learning library that provides implementations of commonly used machine learning algorithms for clustering, classification, regression, collaborative filtering and dimensionality reduction.

Spark GraphX

- Spark GraphX is a component for performing graph computations.
- GraphX provides implementations of common graph algorithms such as PageRank, connected components, triangle counting.

Spark Cluster



Spark Cluster

- Driver:
 - Each Spark application consists of a driver program and is coordinated by a SparkContext object.
- Cluster Manager:
 - Spark supports various cluster managers including Spark's standalone cluster manager, Apache Mesos and Hadoop YARN.
 - The cluster manager allocates resources for applications on the worker nodes.
- Executors:
 - The executors which are allocated on the worker nodes run the application code as multiple tasks.
 - Applications are isolated from each other and run within their own executor processes on the worker nodes.

Spark Context

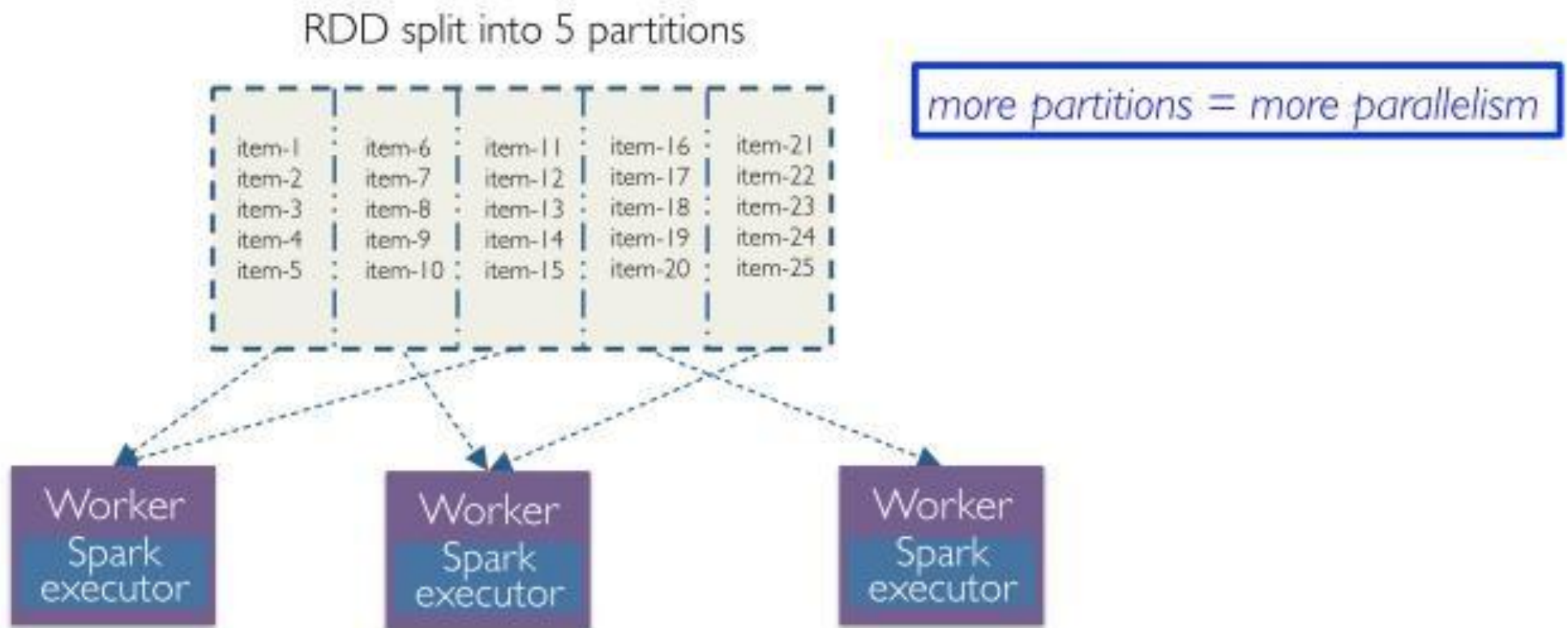
- A Spark program first creates a SparkContext object
 - Tells Spark how and where to access a cluster
- pySpark shell automatically creates the *sc* variable
- Use SparkContext to create RDDs

Resilient Distributed Datasets

- The primary abstraction in Spark
 - Collection of elements partitioned across the nodes of the cluster
 - Immutable once constructed
 - Enable operations on collection of elements in parallel
- You can construct RDDs
 - by parallelizing existing Python collections (lists)
 - by transforming an existing RDDs
 - from files in HDFS or any other storage system

Resilient Distributed Datasets

- Programmer specifies number of partitions for an RDD



pySpark

- Spark supports a shell mode with which you can interactively run commands for analyzing data.
- To launch the Spark Python shell, run the following command:

```
./bin/pyspark
```

Creating RDDs

- RDDs can be created either by parallelizing existing collections or by loading an external dataset

```
■ #Create RDD from a local file
lines = sc.textFile("file:///root/spark/README.md")

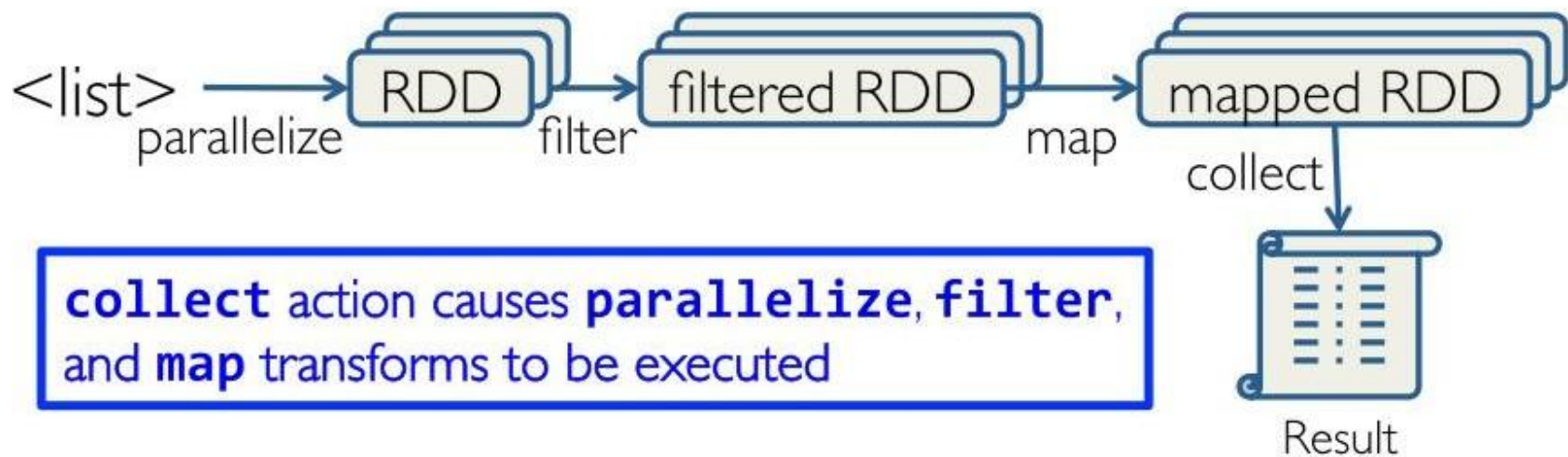
#Create RDD by parallelizing an existing collection
data = sc.parallelize([1, 2, 2, 3, 3, 4, 5])
```


RDD Operations

- Spark RDDs support two types of operations:
 - **Transformations:** Transformations are used to create a new dataset from an existing one.
 - Transformations are lazy (not computed immediately)
 - **Actions:** Actions return a value to the driver program after running a computation on the dataset.
 - Transformed RDD is executed when action runs on it

Working with RDDs

- Create an RDD from a data source
- Apply transformations to an RDD
- Apply actions to an RDD



Transformations

```
■ lines = sc.textFile("file:///root/spark/README.md")  
  
data1 = sc.parallelize([1, 2, 2, 3, 3, 4, 5])  
data2 = sc.parallelize([3, 4, 5, 6, 7, 8])
```

Map: The map transformation takes as input a function which is applied to each element of the dataset, and maps each input item to another item.

```
■ #map transformation example  
lineLengths = lines.map(lambda s: len(s))  
lineLengths.take(5)  
[14, 0, 78, 72, 73]
```

Transformations

Filter: The filter transformation generates a new dataset by filtering the source dataset using the specified function.

```
■ #filter transformation example
filteredLines = lines.filter(lambda line: line.find('Spark')>0)
filteredLines.take(3)
[u'# Apache Spark', u'rich set of higher-level tools including Spark SQL
for SQL and structured', u'and Spark Streaming for stream processing.']
```

Transformations

flatMap: The flatMap transformation takes as input a function which is applied to each element of the dataset. The flatMap transformation can map each input item to zero or more output items.

```
■ #flatMap transformation example
splitLines = lines.flatMap(lambda line: line.split())
splitLines.take(10)
[u' #', u'Apache', u'Spark', u'Spark', u'is',
u'a', u'fast', u'and', u'general', u'cluster']
```

Transformations

reduceByKey: The reduceByKey transformation when applied on dataset containing key-value pairs, aggregates values of each key using the function specified.

```
■ # reduceByKey transformation example
splitLines = lines.flatMap(lambda line: line.split())
words=splitLines.map(lambda word: (word, 1))

counts=words.reduceByKey(lambda a, b: a+b)
counts.take(5)
[(u'all', 1), (u'when', 1), (u'"local"', 1),
(u'including', 3), (u'computation', 1)]
```

Transformations

sample: The sample transformation samples the data with or without replacement.

```
■ #sample transformation example  
datasample = data1.sample(False, 0.5)  
datasample.collect()  
[3, 3, 4]
```

Transformations

union: The union transformation generates a new dataset from the union of two datasets.

```
■ #union transformation example  
data = data1.union(data2)  
data.collect()  
[1, 2, 2, 3, 3, 4, 5, 3, 4, 5, 6, 7, 8]
```


Transformations

intersection: The intersection transformation generates a new dataset from the intersection of two datasets.

```
■ #intersection transformation example  
data = data1.intersection(data2)  
data.collect()  
[4, 5, 3]
```

Transformations

join: The join transformation generates a new dataset by joining two datasets containing key-value pairs.

```
■ #join transformation example
a=sc.parallelize([('John', 1), ('Tom', 2), ('Ben', 3)])
b=sc.parallelize([('John', 'CA'), ('Tom', 'GA'), ('Ben', 'VA')])
c=a.join(b)
c.collect()
[('Ben', (3, 'VA')), ('John', (1, 'CA')), ('Tom', (2, 'GA'))]
```

Actions

reduce: The reduce action aggregates the elements in a dataset using the specified function.

```
■ #reduce transformation example
lineLengths = lines.map(lambda s: len(s))
totalLength = lineLengths.reduce(lambda a, b: a + b)
3526
```

Actions

collect : The collect action is used to return all the elements of the result as an array.

```
■ #Map transformation example  
lineLengths = lines.map(lambda s: len(s))  
lineLengths.collect()  
[14, 0, 78, 72, 73, ..., 70]
```

Actions

count: The count action returns the number of elements in a dataset.

```
■ #count transformation example  
lines.count()  
98
```

Actions

first : The first action returns the first element in a dataset.

```
■ #Map transformation example  
lines.first()  
u' # Apache Spark'
```

Actions

take: The take action returns the the first n elements in a dataset.

```
■ #Map transformation example
```

```
lines.take(3)
```

```
[u'# Apache Spark', u'', u'Spark is a fast and general  
cluster computing system for Big Data.  It provides']
```

Actions

takeSample: The takeSample action returns a sample of containing specified number of elements from a dataset with or without replacement.

```
■ #takeSample transformation example  
data1.takeSample(False, 2)  
[2, 2]
```


Word Count

```
from operator import add
from pyspark import SparkContext

sc = SparkContext(appName="WordCountApp")
lines = sc.textFile("file.txt")
counts = lines.flatMap(lambda x:
x.split(' ')).map(lambda x: (x, 1)).reduceByKey(add)

output = counts.collect()

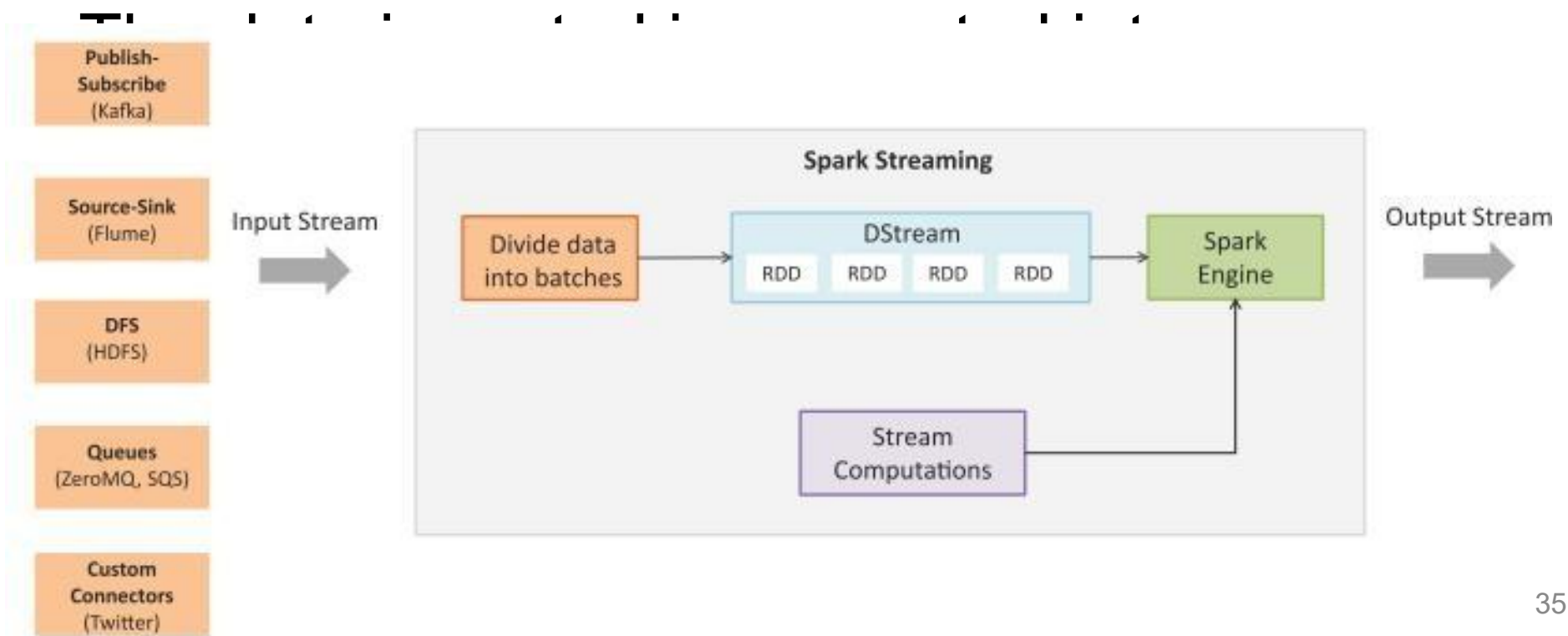
for (word, count) in output:
    print "%s: %i" % (word, count)
```

Spark Streaming

- Spark Streaming is scalable, high throughput and fault tolerant stream processing framework
- Spark Streaming provides a high level abstraction called DStream (discretized stream)
- DStream is a sequence of RDDs

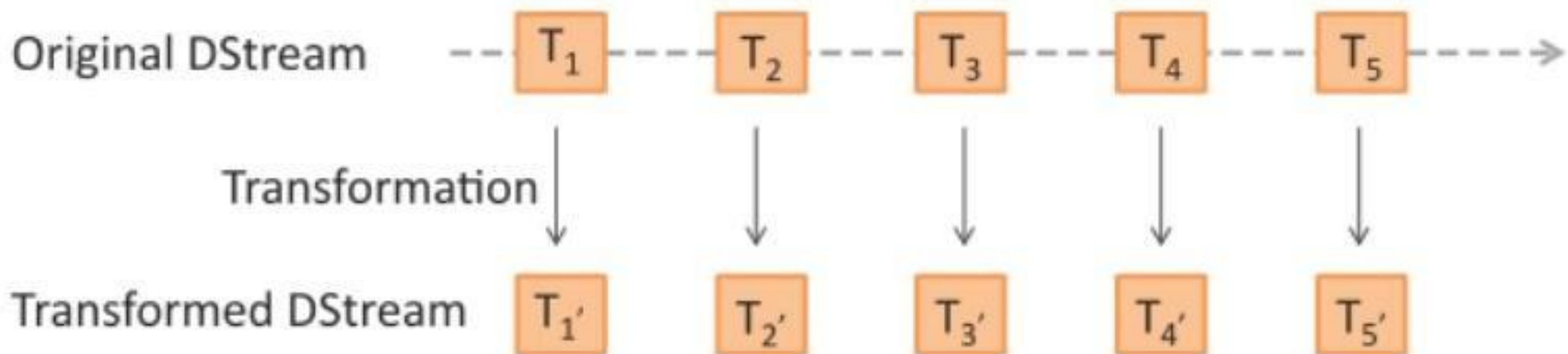
Spark Streaming

- Spark can ingest data from various types of data sources such as publish-subscribe messaging frameworks, messaging queues, distributed file systems and custom connectors.



DStreams

- Just like operations for RDDs, Spark provides operations for DStreams.
- Each RDD contains data from a certain time interval.



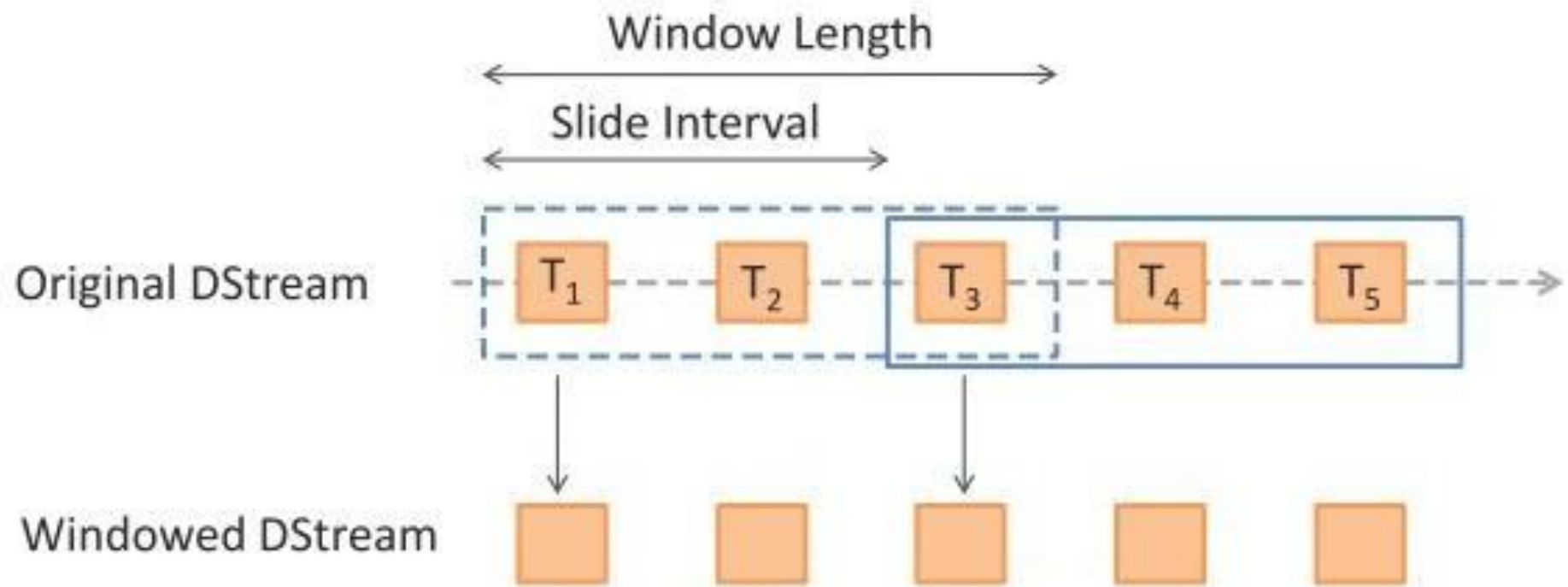
DStream transformations

- The DStream operations are translated into operations on the underlying RDDs.
- DStream transformations such as map, flatMap, filter, reduceByKey are stateless as the transformation are applied to the RDDs in the Dstream separately.

DStream Transformations

- Spark also supports stateful operations such as windowed operations and updateStateByKey operation.
- Stateful operations require checkpointing for fault tolerance purposes.
- For stateful operations, a checkpoint directory is provided to which RDDs are checkpointed periodically.

DStream Window Transformation



Spark MLlib

- Spark MLlib is the Spark's machine learning library which provides implementations of various machine learning algorithms including classification, regression, clustering, collaborative filtering and dimensionality reduction.
- The MLlib APIs are built on top of the Spark's RDDs
- MLlib also provides high-level data types such as Vector, LabeledPoint, Rating and Matrix which are backed by RDDs.

Spark Mllib

Clustering

- k-means
- Gaussian mixture
- Power Iteration Clustering (PIC)
- Latent Dirichlet Allocation (LDA)
- Streaming k-means

Classification & Regression

- SVM
- Logistic Regression
- Linear regression
- Naive Bayes
- Decision Trees
- Random Forests
- Gradient-Boosted Trees
- Isotonic Regression

Collaborative Filtering

- Alternating Least Squares (ALS)

Dimensionality Reduction

- Singular Value Decomposition (SVD)
- Principal Component Analysis (PCA)s

Feature Extraction & Transformation

- TF-IDF
- Word2Vec
- StandardScaler
- Normalizer
- Feature Selection
- Elementwise Product
- PCA

Frequent Pattern Mining

- FP-growth
- Association Rules
- PrefixSpan

Optimization

- Stochastic Gradient Descent
- Limited-memory BFGS (L-BFGS)

Statistics

- Summary Statistics
- Correlations
- Stratified Sampling
- Hypothesis Testing
- Random Data Generation

Evaluation Metrics

- Precision
- Recall
- F-measure
- ROC
- Area Under ROC Curve
- Area Under Precision-Recall Curve

```
#Data format:
#1.0,26.0,36.0,2860.0,274.0

import numpy as np
from pyspark import SparkContext

from pyspark.mllib.regression import LabeledPoint
from pyspark.mllib.classification import NaiveBayes

# Parse a line of text into an MLib LabeledPoint object
def parsePoint(line):
    values = [float(s) for s in line.split(',') ]
    return LabeledPoint(values[0], values[1:])

sc = SparkContext(appName="App")
points = sc.textFile("nbdata.txt").map(parsePoint)

# Train a naive Bayes model.
model = NaiveBayes.train(points, 1.0)

# Make prediction.
prediction = model.predict([20.0, 40.0, 1000.0, 300.0])
print "Predition is: " + str(prediction)
```

SparkSQL

- Spark SQL is a component of Spark which enables interactive querying.
- Spark SQL is useful for querying structured and semi-structured data using SQL-like queries.
- Spark SQL provides a programming abstraction called DataFrames.
- A DataFrame is a distributed collection of data organized into named columns.
- DataFrames can be created from existing RDDs, structured data files (such as text files, Parquet, JSON, Apache Avro), Hive tables and also from external databases.
- Spark provides a Data Sources API which allows accessing structured data through Spark SQL.

SparkSQL

```
■ lines = sc.textFile("file:///home/hadoop/
googlebooks-eng-us-all-2gram-20090715-50.csv")

parts = lines.map(lambda l: l.split("\t"))

ngrams = parts.map(lambda x: Row(ngram=x[0],
year=int(x[1]), ngramcount=int(x[2]), pages=int(x[3]), books=int(x[4])))

schemaNGrams = sqlContext.createDataFrame(ngrams)
```

```
■ >>> schemaNGrams.show()
```

```
+---+-----+-----+---+---+
|books| ngram|ngramcount|pages|year|
+---+-----+-----+---+---+
|  1|  !  09|          1|  1|1829|
|  3|  !  09|          3|  3|1879|
|  2|  !  09|          2|  2|1911|
|  4|  !  09|          4|  4|1941|
```

SparkSQL

```
■ schemaNGrams.filter(schemaNGrams['ngramcount'] > 5).show()
```

```
+---+-----+-----+---+---+
|books| ngram|ngramcount|pages|year|
+---+-----+-----+---+---+
| 12| ! 09| 17| 17|1994|
| 2| ! 1944| 8| 8|1977|
| 11| ! 28| 15| 15|1866|
| 10| ! 28| 10| 10|1891|
| 32| ! 28| 37| 37|1916|
```

```
■ schemaNGrams.groupBy("year").count().show()
```

```
+---+-----+
|year|count|
+---+-----+
|1831| 79|
|1832| 57|
|1833| 56|
```


SparkSQL

```
■ schemaNGrams.registerTempTable("ngrams")
```

```
result = sqlContext.sql("SELECT ngram, ngramcount  
FROM ngrams WHERE ngramcount >= 5").show()
```

```
+-----+-----+  
| ngram|ngramcount|  
+-----+-----+  
| ! 09| 17|  
| ! 1944| 8|  
| ! 28| 15|
```

```
■ result = sqlContext.sql("SELECT ngram, year, MAX(ngramcount) AS  
maxCount FROM ngrams GROUP BY ngram, year").show()
```